

Lecture 9: CPU Registers

Learning Objectives

By the end of this lecture, students will be able to:

- Define CPU Registers.
 - Explain why registers are the fastest memory in a computer.
 - Differentiate between General-Purpose and Special-Purpose Registers.
 - Explain the functions of the Accumulator, Program Counter (PC), Instruction Register (IR), Stack Pointer (SP), Base Register (BR), Index Register (IX), and Flag Register.
 - Understand register transfer operations.
 - Understand how registers are used in Assembly Language.
 - Compare Registers, Cache, RAM, and Secondary Storage.
-

1. Introduction

Teaching Script

First 30-Second SEO Phrase

Naturally say:

"In today's Computer Organization and Assembly Language lecture, we'll study CPU Registers, including General-Purpose Registers, Program Counter (PC), Instruction Register (IR), Stack Pointer (SP), Flag Register, Base Register, and Index Register, with simple examples and assembly language concepts."

In our previous lecture, we studied the Control Unit and learned how it controls the execution of instructions using components like the Program Counter, Instruction Register, MAR, and MDR.

Today, we're going to study one of the fastest and most important storage components inside the CPU—**Registers**.

Whenever a processor executes an instruction, it rarely works directly with RAM. Instead, it first loads data into registers because registers are much faster than main memory.

In Assembly Language, almost every instruction uses registers. That is why understanding registers is essential before we begin writing assembly programs later in this course.

Let's get started.

2. What Are CPU Registers?

Definition

Registers are small, high-speed storage locations inside the CPU that temporarily store data, instructions, addresses, and intermediate results during program execution.

In simple words:

Registers are the CPU's working memory.

They are located **inside the processor**, making them much faster than RAM.

3. Why Are Registers Needed?

Imagine solving a math problem.

If you had to open a cupboard every time you needed a number, you'd waste time.

Instead, you keep the numbers on a piece of paper in front of you.

That paper acts like a **register**.

Similarly:

- **RAM** stores large amounts of data.
 - **Registers** store only the values the CPU needs immediately.
-

4. Memory Hierarchy

Registers are the fastest storage in a computer.

Fastest

Registers

Cache Memory

RAM

SSD / Hard Disk

Slowest

Characteristics

Storage	Speed	Capacity	Location
---------	-------	----------	----------

Registers	Fastest	Very Small	Inside CPU
Cache	Very Fast	Small	CPU / Near CPU
RAM	Moderate	Large	Main Memory
SSD/HDD	Slowest	Very Large	Secondary Storage

5. Types of Registers

Registers are broadly divided into two categories:

CPU Registers

|

└─ General-Purpose Registers

|

└─ Special-Purpose Registers

6. General-Purpose Registers (GPR)

These registers are used by programmers and compilers to store temporary values.

They can hold:

- Numbers
- Variables
- Intermediate results
- Addresses (depending on the instruction set)

Example (conceptual):

R1 = 10

R2 = 20

The ALU adds them:

R1 + R2

↓

30

Stored in:

R3 = 30

In MIPS, examples include registers like `$t0`, `$t1`, and `$s0`.

7. Special-Purpose Registers

These registers perform dedicated tasks.

Common special-purpose registers include:

- Accumulator (in some architectures)
 - Program Counter (PC)
 - Instruction Register (IR)
 - Stack Pointer (SP)
 - Flag/Status Register
 - Base Register (BR)
 - Index Register (IX)
-

8. Accumulator Register (ACC)

The **Accumulator** stores intermediate arithmetic and logical results.

Example:

10 + 20

↓

30

↓

Accumulator

Many classic processors used an accumulator extensively. Modern architectures often rely more on general-purpose registers.

9. Program Counter (PC)

We briefly discussed the Program Counter in the previous lecture.

Function

Stores the **address of the next instruction** to be executed.

Example:

Memory

1000 → LOAD

1001 → ADD

1002 → STORE

Initially:

PC = 1000

After fetching:

PC = 1001

The PC automatically updates unless a jump or branch instruction changes its value.

10. Instruction Register (IR)

The IR stores the instruction currently being executed.

Example:

Instruction

ADD R1, R2

The Control Unit decodes this instruction from the IR.

11. Stack Pointer (SP)

The **Stack Pointer** stores the address of the top of the stack.

Stacks are used for:

- Function calls
- Returning from functions
- Saving registers
- Local variables

Example:

Stack

↓

Top



SP

Whenever data is pushed or popped, the Stack Pointer changes accordingly.

12. Base Register (BR)

The Base Register stores a **base address**.

Programs can access memory relative to this base.

Example:

Base Register = 1000

Offset = 25

Actual Address = 1025

This technique is useful for accessing arrays, program segments, and relocated memory.

13. Index Register (IX)

The Index Register is commonly used when accessing arrays.

Example:

A[5]

Suppose:

Base Address = 1000

Index = 5

Element Size = 4 bytes

Address calculation:

$1000 + (5 \times 4)$

↓

1020

The CPU uses the base and index values to locate the desired element.

14. Flag (Status) Register

The Flag Register stores information about the result of ALU operations.

Common flags include:

- Zero (Z)
- Carry (C)
- Sign/Negative (S or N)
- Overflow (V)

Example:

$$5 - 5 = 0$$

Zero Flag becomes **1**.

These flags are important for branch instructions in assembly language.

15. Register Transfer Operations

Registers exchange data using register transfer operations.

Example:

R1 = 10

↓

Move

↓

R2 = 10

Conceptually:

$$R2 \leftarrow R1$$

This means the contents of **R1** are copied into **R2**.

16. Registers in Assembly Language

Suppose we want to add two numbers.

Conceptual MIPS example:

```
add $t0, $t1, $t2
```

Meaning:

$$\$t0 = \$t1 + \$t2$$

Registers involved:

- **\$t1** → First operand
- **\$t2** → Second operand
- **\$t0** → Result

Notice that the ALU operates on values already stored in registers.

17. How Registers Work Together

Example:

```
int c = a + b;
```

Execution:

RAM



Registers



ALU



Registers



RAM

Registers act as temporary workspaces for the CPU.

18. Complete Register Organization

CPU

+-----+

| Control Unit |

+-----+

+-----+

| ALU |

+-----+

+-----+

| Registers |

|-----|

| GPR |

| PC |

| IR |

| SP |

| BR |

| IX |

| Flag Register |

+-----+

19. Real-Life Analogy

Imagine you're preparing a meal.

- **Kitchen Cabinet** → RAM (stores many ingredients)
- **Kitchen Counter** → Registers (ingredients you're using right now)
- **Chef** → ALU
- **Restaurant Manager** → Control Unit

The chef doesn't repeatedly open the cabinet for every ingredient. Instead, the needed ingredients are placed on the counter first.

Likewise, the CPU loads values into registers before processing them.

20. Practical Example

Suppose:

`int x = 8;`

`int y = 4;`

`int z = x + y;`

Execution:

Memory

↓

R1 = 8

R2 = 4



ALU



12



R3



Memory



Lecture Summary

Today, we explored CPU Registers, the fastest storage locations inside the processor.

We learned the difference between General-Purpose and Special-Purpose Registers and studied the functions of the Accumulator, Program Counter, Instruction Register, Stack Pointer, Base Register, Index Register, and Flag Register.

We also saw how registers work together with the Control Unit and ALU to execute instructions efficiently and how assembly language relies heavily on register operations.

In the next lecture, we'll study the **System Bus**, including the **Data Bus**, **Address Bus**, and **Control Bus**, and understand how the CPU communicates with memory and input/output devices.

Thank you for watching, and I'll see you in the next lecture.
